

Computer Programming 143 – Lecture 16

Arrays IV

Faculty of Engineering



Copyright

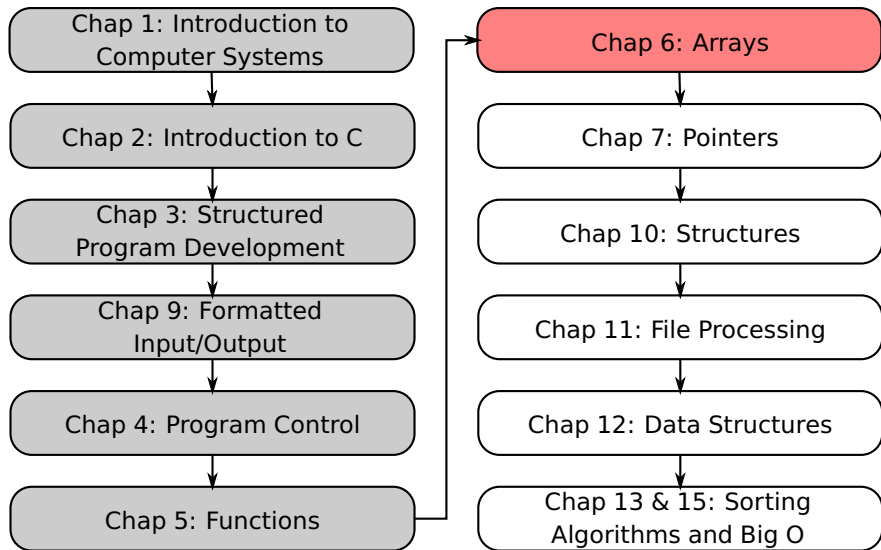
Copyright © 2026 Stellenbosch University
All rights reserved

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

Module Overview



Lecture Overview

1 Multiple-Subscripted Arrays (6.11)

2 Debugging

6.11 Multiple-Subscripted Arrays

Multiple-Subscripted Arrays

- Tables with rows and columns (**m** by **n** array)
- Like matrices: specify row, then column

	col 0	col 1	col 2	col 3
row 0	c[0][0]	c[0][1]	c[0][2]	c[0][3]
row 1	c[1][0]	c[1][1]	c[1][2]	c[1][3]
row 2	c[2][0]	c[2][1]	c[2][2]	c[2][3]
row 3	c[3][0]	c[3][1]	c[3][2]	c[3][3]

Diagram illustrating the structure of a multiple-subscripted array (matrix) with 4 rows and 4 columns. The array is represented as a table with rows and columns labeled. The array name 'c' is shown in the first subscript of the element c[3][1]. The row subscript '3' is shown in the second subscript of the element c[3][1]. The column subscript '1' is shown in the third subscript of the element c[3][1].

Array name Row subscript Column subscript

6.11 Multiple-Subscripted Arrays

Example 1

```
// fig06_16.c
// Initializing multidimensional arrays.
#include <stdio.h>
void printArray(int a[][3]);
int main(void) {
    int array1[2][3] = {{1, 2, 3}, {4, 5, 6}};
    printf("Values in array1 by row are:\n");
    printArray(array1);
    int array2[2][3] = {{1, 2, 3}, {4, 5}};
    printf("\nValues in array2 by row are:\n");
    printArray(array2);
    int array3[2][3] = {{1, 2}, {4}};
    printf("\nValues in array3 by row are:\n");
    printArray(array3);
    return 0;
}
```

6.11 Multiple-Subscripted Arrays

Example 1 (cont...)

```
// function to output array with two rows and three columns
void printArray(int a[][3]) {
    // loop through rows
    for (size_t i = 0; i <= 1; ++i) {
        // output column values
        for (size_t j = 0; j <= 2; ++j) {
            printf("%d ", a[i][j]);
        }
        printf("\n"); // start new line of output
    }
}
```

6.11 Multiple-Subscripted Arrays

Example 2 (main)

```
// fig06_17.c
#include <stdio.h>
#define STUDENTS 3
#define EXAMS 4

int minimum(const int grades[][EXAMS], size_t pupils, size_t tests);
int maximum(const int grades[][EXAMS], size_t pupils, size_t tests);
double average(const int setOfGrades[], size_t tests);
void printArray(const int grades[][EXAMS], size_t pupils, size_t tests);

int main(void) {
    // initialize student grades for three students (rows)
    int studentGrades[STUDENTS][EXAMS] =
        {{77, 68, 86, 73},
         {96, 87, 89, 78},
         {70, 90, 86, 81}};
```

6.11 Multiple-Subscripted Arrays

Example 2 (main cont. . .)

```
// output array studentGrades
printf("The array is:\n");
printArray(studentGrades, STUDENTS, EXAMS);
// determine smallest and largest grade values
printf("\n\nLowest grade: %d\nHighest grade: %d\n",
minimum(studentGrades, STUDENTS, EXAMS),
maximum(studentGrades, STUDENTS, EXAMS));
// calculate average grade for each student
for (size_t student = 0; student < STUDENTS; ++student) {
    printf("The average grade for student %zu is %.2f\n",
        student, average(studentGrades[student], EXAMS));
}
return 0;
}
```

6.11 Multiple-Subscripted Arrays

Example 2 (cont...)

```
// Find the minimum grade
int minimum(const int grades[][EXAMS], size_t pupils, size_t tests) {
    int lowGrade = 100; // initialize to highest possible grade
    // loop through rows of grades
    for (size_t row = 0; row < pupils; ++row) {
        // loop through columns of grades
        for (size_t column = 0; column < tests; ++column) {
            if (grades[row][column] < lowGrade) {
                lowGrade = grades[row][column];
            }
        }
    }
    return lowGrade; // return minimum grade
}
```

6.11 Multiple-Subscripted Arrays

Example 2 (cont...)

```
// Find the maximum grade
int maximum(const int grades[][EXAMS], size_t pupils, size_t tests) {
    int highGrade = 0; // initialize to lowest possible grade
    // loop through rows of grades
    for (size_t row = 0; row < pupils; ++row) {
        // loop through columns of grades
        for (size_t column = 0; column < tests; ++column) {
            if (grades[row][column] > highGrade) {
                highGrade = grades[row][column];
            }
        }
    }
    return highGrade; // return maximum grade
}
```

6.11 Multiple-Subscripted Arrays

Example 2 (cont...)

// Determine the average grade for a particular student

```
double average(const int setOfGrades[], size_t tests) {
```

```
    int total = 0; // sum of test grades
```

```
    // total all grades for one student
```

```
    for (size_t test = 0; test < tests; ++test) {
```

```
        total += setOfGrades[test];
```

```
    }
```

```
    return (double) total / tests; // average
```

```
}
```

// Print the array

```
void printArray(const int grades[][EXAMS], size_t pupils, size_t tests) {
```

```
    // output column heads
```

```
    printf("%s", "                                [0] [1] [2] [3]");
```

```
    // output grades in tabular format
```

6.11 Multiple-Subscripted Arrays

Example 2 (cont...)

```
for (size_t row = 0; row < pupils; ++row) {  
    // output label for row  
    printf("\nstudentGrades[%zu] ", row);  
    // output grades for one student  
    for (size_t column = 0; column < tests; ++column) {  
        printf("%-5d", grades[row][column]);  
    }  
}
```

6.11 Multiple-Subscripted Arrays

Initialisation

```
int b[2][2] = { { 1, 2 }, { 3, 4 } };
```

- Initialisers grouped by row in braces
- If not enough, unspecified elements set to zero

```
int b[2][2] = { { 1 }, { 3, 4 } };
```

1	2
3	4

1	0
3	4

6.11 Multiple-Subscripted Arrays

Referencing elements

- Specify row, then column

```
printf( "%d", b[0][1] );
```

Refer to Fig. 6.16 and 6.17 in Deitel & Deitel for examples

What is debugging?

- It is a systematic process of identifying and fixing errors (bugs) in a computer program
 - Errors may lead to wrong results or code that does not compile

Debugging methods

Using printf () statements

- Print out the data in a particular variable to see if it is as expected
- To see if a certain line is executed:
`printf("%s:%d\n", __FILE__, __LINE__);`
- To interrupt the program at specific point:
`exit(-1);` *// defined in stdlib.h library*

Use /*.....*/ comment blocks

- Block out large sections that may be leading to an error
- Unblock single statements / small sections until the error occurs

Best practice - use the debugger

- Those red arrows etc at the right top in Code::Blocks
- Install if not available - see STEMLearn

Today

- Multiple-subscripted arrays
- Debugging

Next

- Pointers

Homework

- 1 Study Section 6.11 in Deitel & Deitel
- 2 Do Self Review Exercise 6.4 in Deitel & Deitel
- 3 Do Exercises 6.8(d) and 6.21 in Deitel & Deitel